Space, Magic and Mutants

Memoria de Desarrollo

git@github.com:Josemi2005HXC/SpaceMagicandMutants.git

AUTOR/A	Josef Carbonell Ventura
ASIGNATURA	Progradation
FECHA	13/11/2026

ÍNDICE

1. Descripción 3

2. Funcionalidades 4

3. Diseño 4

4. Pruebas 11

5. Dificultades 15

1. Descripción:

El objetivo es derrotar enemigos, se gana cuando bajas la vida del enemigo a 0, si pierdes todos tus puntos de vida entonces se acaba y pierdes la partida, también puedes abandonar después de derrotar a cada enemigo.

Cada uno (jugador como enemigo) contáis con una acción por turno, el jugador puede seleccionar que hacer (atacar, ponerse en posición de defensa o huir), la acción de huir funciona de forma aleatoria, puede huir con éxito o puede no tener éxito y exponerse a otro ataque enemigo.

La vida del jugador se verá reducida en cada combate según el daño que haya recibido.

2. Funcionalidades:

No se ha implementado exploración o más interacciones con otros personajes ni tampoco ítem y recursos que encontrar explorando.

De momento estas son las funcionalidades:

Funcionalidad	Descripción	Funcionamiento
Creacion de personaje	Permite crear nuestro personaje con su nombre, especie y rol.	El jugador debe introducir un nombre y una especie y rol validos
Roles y especie	Determina nuestras estadísticas iniciales.	Mediante un switch el sistema comprueba que bonificaciones y penalizaciones otorgar según rol y especie, también podrá hacer que según el rol o especie tengamos un mensaje u otro de victoria.
Combates	Permite enfrentarse por turnos a distintos enemigos.	Mediante una opción boolean e un if nos permite combatir mientras nos queden puntos de vida
Enemigos aleatorios	En cada combate nos enfrentaremos a distintos enemigos con estadísticas diferentes.	Mediante un .random y for recorremos el array de enemigos para seleccionar uno distinto en cada combate, también .random le asigna diferentes estadísticas.
Ataque	En el combate podremos causar daño al enemigo.	Calculamos el daño del ataque teniendo en cuenta la vida, nuestro ataque y su defensa.
Defensa	Podemos aumentar nuestra defensa poniéndonos en posición para no sufrir daño.	Se suman 5 puntos de defensa y a la hora de recibir daño, este se amortigua.
Huir	Nos permite escapar del combate.	Mediante un .random y un if determinamos que se pueda o no huir.
Menús	Podremos seleccionar varias opciones al iniciar el juego.	Un pequeño menú que pueda servir para moverse y empezar a jugar como a ver la guía.
Victoria y derrota	Determina si hemos ganado o perdido	Comprueba los puntos de vida y en base a eso determina si sigue la partida, se pierde o se vence al enemigo.

3. Diseño:

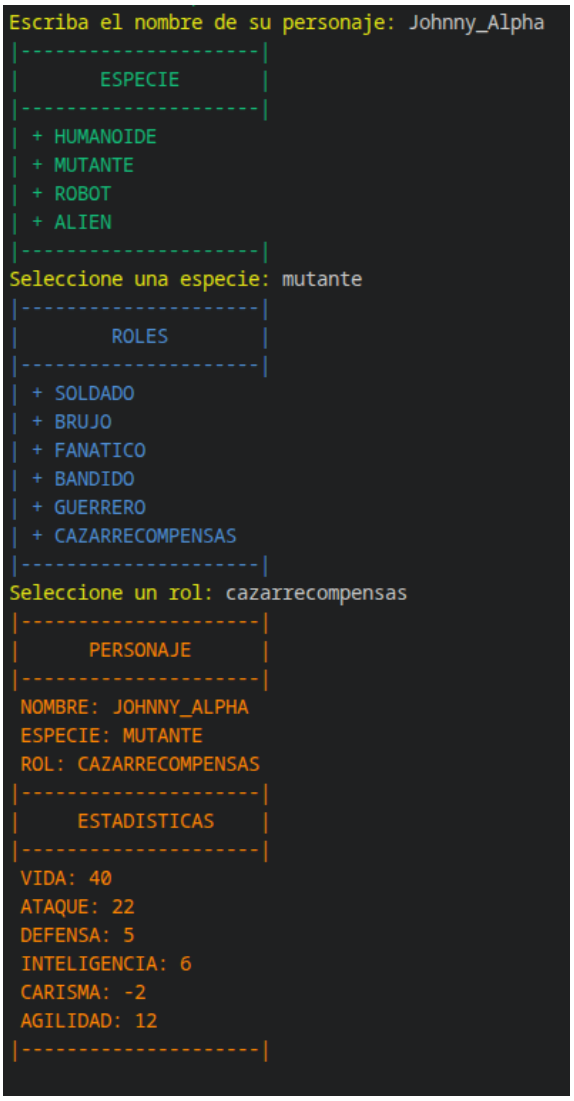
Tabla de implementaciones:

Tipo	Dato/Utilidad	Descripción
final	Colores ANSII	Convierte variables en constantes, en nuestro caso los String que contienen códigos ANSII.
int	Números	Nos sirve para almacenar los números de variables (puntos de estadísticas, opciones de switch.
String	Cadenas de texto	Almacenamos datos de tipo texto (nombres de elementos en los arrays, nuestro nombre).
switch	Casos	Almacena las instrucciones que ejecutara según la opción que seleccionemos o si coincide alguna opción (ejemplo rol coincide con “Brujo” entonces suma y quita x cantidad de atributos).
If, else, else if	Ejecución o salida	Ejecutara algo o saldrá de un bucle si se cumplen unas condiciones (coincidencias, no coincidencias, true/false)
boolean	True/False	Lo utilizamos para comprobar si la partida sigue, has ganado o has perdido.
.equals	Comprobación	Sirve para verificar

		coincidencias entre Strings.
.random	Generar números aleatorios	Nos sirve para que los atributos y ataques sean aleatorios y seleccionar dentro del array un enemigo aleatorio
for	Repetir	Sirve para que recorramos arrays hasta el final
Array unidimensionales	Listas de elementos	Almacenamos una lista de elementos, en nuestro caso almacenamos enemigos, razas, y especies.
Array.atList	Comprobación	Sirve para comprobar que en los arrays está un dato.
.toUpper	Transformar	Convertimos las minúsculas de los datos que ingresamos en mayúsculas para que coincidan con el contenido del Array.
Do while	Repetir hasta	Con Do indicamos que va a realizar y con while las condiciones que debe cumplir para parar y salir del bucle.
print y println	Salida	Damos salida a los datos y mostramos los mensajes que queremos.

4. Pruebas:

Nº	Caso	Resultado esperado	Resultado obtenido	
01	Se crea el personaje con nombre y un rol y especie existente en el array	Que permita crearlo correctamente y aparezca su ficha de personaje	El personaje se crea correctamente y aparece su ficha de personaje	✓
02	Se introduce un elemento que no está en el array especie	Que se muestre un mensaje de error y el bucle se repita.	El bucle se vuelve a repetir con el mensaje de error.	✓
03	Empezar combate	Que empiece el combate después de crear un personaje.	Empieza el combate y no da error.	✓
04	Introducir datos en minúsculas	Que el juego acepte minúsculas sin problema.	Acepta minúsculas y no da errores.	✓
05	Atacar	Que permita atacar y le reste una cantidad de vida correspondiente.	El enemigo pierde la cantidad de vida que le corresponde.	✓
06	Seleccionar una acción errónea	Que al seleccionar una acción errónea, el bucle se repita y no pase al turno enemigo.	El bucle se repite y no deja pasar al turno enemigo.	✓
07	Salir	En el menú inicial poder salir.	Se puede salir perfectamente y se muestra el mensaje de despedida.	✓
08	Derrotar enemigo	Que termine el combate enemigo y salte el mensaje de victoria en caso de haber.	El combate con el enemigo termina y cuando lo ahí aparece el mensaje de victoria.	✓



5. Dificultades:

Se intento crear los roles y especies en un principio sin arrays, despues se solucionó precisamente usando arrays, ya que no permita tanta flexibilidad y dificultaba las cosas (los prints eran más largos al no poder usar for para recorrer y aparte no poderse establecer unos roles y especies exactos que debían estar en el array correspondiente y valiéndose introducir cosas

incorrectas y liándome con el if y while).

Por errores de sintaxis, se pensaba que con Switch no se podía establecer valores a cada clase de enemigo, se corrigieron y si se pudo.

Aunque se añadió agilidad como estadística debido a errores no se supo cómo hacer que para huir agilidad sea lo que se mida.

En futuras actualizaciones se deben meter más mensajes, más capas para la personalización, cambiar aliens por especies exactas y meter tipos de mutantes (psíquicos, animales mutantes, forzudos…) aparte meter sub-roles como brujo y
dentro (brujo de las cartas, brujo de las invocaciones, brujo de los instrumentos…).

También deberíamos profundizar más el combate, en vez de solo atacar poder hacer distintos ataques, el soldado podría disparar o usar una granada, o pegar una patada con su bota, el brujo podría usar su espada para atacar o usar sus magias, si es mago de las cartas, tener una acción de sacar una carta aleatoria de su baraja del Tarot y según cual sea obtener distintos bonos.

También se debería añadir estados como confusión, aturdimiento, maldición… aparte de daño crítico y probabilidad de esquivar ataques.

También, si es posible meter exploración e interacciones ya que carisma esta también añadida, pero de momento no tiene uso.